Enhancing metaheuristics for pistachio supply chain optimization

Raissa Gonçalves Diniz[1] and André Costa Batista[1,2]

[1]Operations Research and Complex Systems Laboratory, Universidade Federal de Minas Gerais, Brazil.
[2]Department of Electrical Engineering, Universidade Federal de Minas Gerais, Av. Antônio Carlos 6627, Belo Horizonte, 31270-010, Minas Gerais, Brazil.

Contributing authors: raissagd@ufmg.br; andre-costa@ufmg.br;

Abstract

Supply chains are vital for the agricultural sector, particularly for high-value crops like pistachios, where efficient management can enhance both economic growth and sustainability. Although there has been research on optimizing pistachio supply chains, most studies rely heavily on population-based algorithms to solve these problems. In this study, we propose to apply a Variable Neighborhood Search (VNS) algorithm and comparing its performance against a population-based metaheuristic (Genetic Algorithm, GA) and an exact method. Our findings reveal that VNS consistently outperforms GA in both execution time and objective function value, particularly for larger instances. Additionally, despite a slight compromise in the final solution evaluation, VNS reduces execution time by nearly 97% compared to the exact method in some of the larger instances. Therefore, the proposed method demonstrates its efficiency to address supply chain optimization problems.

Keywords: Supply Chain Optimization, Metaheuristics, Variable Neighborhood Search, Sustainability

1 Introduction

The growing role of agriculture in the economic development of many countries highlights the importance of integrating this sector with other industries to stimulate

overall economic growth. Agriculture not only supports food security but also acts as a major contributor to exports, generating significant income and bolstering other sectors. Among various agricultural products, pistachios stand out due to their high economic and nutritional value. Major producers such as the US, Turkey, and Iran dominate global pistachio production, remaining the leading producers worldwide in 2022 and 2023 [1]. As a result, the strategic importance of pistachios in international trade continues to grow.

With the increasing global demand for pistachios, the need for efficient supply chain networks becomes more critical. Proper supply chain management is vital for maintaining product quality, minimizing environmental impacts, and ensuring the smooth flow of goods from production to consumers. A key challenge in pistachio production is the management of its by-products, such as hulls and shells, which, while free of heavy metals or pathogens, can degrade soil quality and contribute to environmental pollution if not properly handled [2, 3]. Furthermore, improper disposal of these materials may lead to the growth of molds that increase the risk of aflatoxin contamination, posing serious health risks such as liver cancer [4, 5]. Therefore, eco-friendly practices for reusing agricultural waste are essential to mitigate both environmental and health hazards.

Moreover, inefficiencies in supply chain networks for food and agricultural products, including pistachios, can have significant implications for product quality and trade [6]. Addressing these environmental and logistical concerns is crucial to sustaining both the economic and ecological viability of the pistachio industry. In the broader context of sustainable supply chains, several studies have focused on optimizing resource use and minimizing waste through recovery and recycling practices. For example, sustainable supply chain models designed for agriculture and other industries have shown significant environmental benefits by reducing waste and reusing by-products for processes like biogas and compost production [7]. Such approaches have been successfully applied in sectors such as manufacturing and waste recovery, where efficient resource management and cost reduction have been key drivers [8, 9]. These strategies underline the importance of sustainability in modern supply chains and the potential for agricultural industries to implement similar measures.

In related research, Rajabi-Kafshgar et al. [10] focused on optimizing the pistachio supply chain using various algorithms, primarily population-based approaches such as Genetic Algorithms (GA) [11, 12], among others. Although the authors executed each algorithm a reasonable number of times, the paper provides limited evidence of any algorithm demonstrating statistically superior performance, as their comparisons are based on the best solutions found by each method. Additionally, some of the graphs in the article are not adequately edited, which may hinder the clarity of the presented results (see, for example, Fig. 15 in [10]).

Furthermore, there is ongoing debate about whether the differences in bio-inspirations behind metaheuristics, such as the Imperialist Competitive Algorithm, the Keshtel Algorithm, and the Social Engineering Optimizer – all of which were used by the authors – can indeed lead to significant performance differences [13, 14]. This raises questions about the practical impact of such design choices in algorithmic performance.

Given the complex interplay of factors influencing pistachio production and trade, and building upon this previous work, this research focuses on designing an optimal supply chain network for pistachios by leveraging the Variable Neighborhood Search (VNS) algorithm [15]. VNS is particularly well-suited for large combinatorial optimization problems and it typically involve few parameters [16]. We address the mixed linear model of [10], which integrates forward and backward logistics for efficient resource management, including the reuse of by-products through composting. To reduce costs and environmental impact, we apply VNS and compare its effectiveness against GA and an exact method. Our findings highlight evidences for VNS's superior performance compared to GA in terms of objective function evaluation. It also achieves reasonable solutions while requiring significantly less computational time than the exact method for large-scale problems. This demonstrates its potential as a scalable and efficient tool for sustainable supply chain management in agriculture.

The remainder of this paper is organized as follows. In Section 2, we present the problem definition, describing the pistachio supply chain network, its components, and the cost-minimization objectives under capacity and demand constraints. Section 3 focuses on the Heuristic Approach, where we introduce the Variable Neighborhood Search (VNS) algorithm, along with its solution representation using priority-based encoding, neighborhood structures, and algorithmic details. In Section 4, we detail the results of our experiments, comparing VNS with both an exact method and a population-based algorithm (GA). Finally, Section 5 presents the Conclusion, summarizing the key findings of the study, discussing the advantages of VNS for pistachio supply chain optimization, and suggesting future research directions for improving solution efficiency and applicability in broader contexts.

2 Problem definition

The network proposed by [10], as illustrated in Figure 1, includes producers, processing centers, pistachio factories, oil extraction centers, composting centers, and customers. Pistachios are transported from orchards to processing centers, producing open-mouth pistachios, raw kernels, and waste, which are then sent to factories, oil extraction centers, and composting centers, respectively. Open-mouth pistachios are roasted, packed, and shipped to customers, while pistachio oil is delivered to oil customers and cosmetic factories. Waste is converted into compost for distribution. Although the compost returns to the producers, ideally forming a closed-loop in the supply chain, mathematically, there is no connection between the compost customers and the producers who send the pistachios to the processing centers. In other words, the mathematical model of the flow does not implement a closed-loop, strictly speaking. It only interprets the flow of compost to its respective customers as feedback, even though these customers are not necessarily related to the variables associated with the producers. This has been done in other studies [17–20], and the strict implementation of a feedback loop could be a next step for research.

The mixed-integer linear problem aims to minimize total costs (1), including opening, transportation, and production costs, under the following capacity and demand

constraints with no allowed shortages (see further details regarding the variables in the Table 5 of [10]):

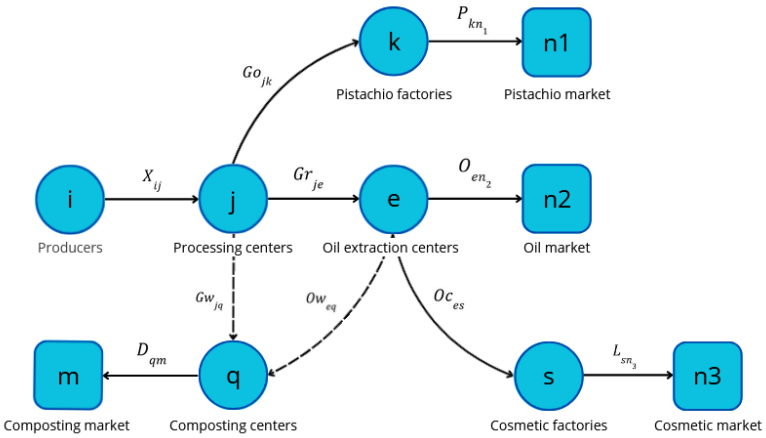


Fig. 1: Proposed pistachio supply chain network

$$\min \; z_1 + z_2 + z_3 \tag{1}$$

$$\text{s.t.:} \quad \sum_{j=1}^{J} X_{ij} \leq Cpa_i, \quad \forall i \in I \tag{2}$$

$$\sum_{i=1}^{I} X_{ij} \leq Cpu_j \times U_j, \quad \forall j \in J \tag{3}$$

$$\sum_{e=1}^{E} Ow_{eq} + \sum_{j=1}^{J} Gw_{jq} \leq Cpy_q \times Y_q, \quad \forall q \in Q \tag{4}$$

$$\sum_{j=1}^{J} Go_{jk} \leq Cpw_k \times W_k, \quad \forall k \in K \tag{5}$$

$$\sum_{j=1}^{J} Gr_{je} \leq Cpr_e \times R_e, \quad \forall e \in E \tag{6}$$

$$\sum_{e=1}^{E} O_{en_2} \leq Cpv_s \times V_s, \quad \forall s \in S \tag{7}$$

$$\sum_{k=1}^{K} Go_{jk} + \sum_{e=1} Gr_{je} + \sum_{q=1}^{Q} Gw_{jq} \leq \sum_{i=1} X_{ij}, \quad \forall j \in J \tag{8}$$

$$\sum_{k=1}^{K} Go_{jk} = (1-\beta)\sum_{i=1}^{I} X_{ij}\theta_1, \quad \forall j \in J \tag{9}$$

$$\sum_{e=1}^{K} Gr_{je} = (1-\beta)\sum_{i=1}^{I} X_{ij}\theta_2, \quad \forall j \in J \tag{10}$$

$$\sum_{q=1}^{Q} Gw_{jq} = \sum_{i=1}^{I} X_{ij}\theta_3, \quad \forall j \in J \tag{11}$$

$$\theta_1 + \theta_2 + \theta_3 = 1 \tag{12}$$

$$\sum_{n_1=1}^{N_1} P_{kn_1} \leq \sum_{j=1}^{J} Go_{jk} \times \gamma k, \quad \forall k \in K \tag{13}$$

$$\sum_{n_2=1}^{N_2} O_{en_2} + \sum_{s=1}^{S} Oc_{es} \leq \sum_{j=1} Gr_{je} \times (1-\lambda), \quad \forall e \in E \tag{14}$$

$$\sum_{q=1}^{Q} Ow_{eq} \leq \sum_{j=1}^{J} Gr_{je} \times \lambda, \quad \forall e \in E \tag{15}$$

$$\sum_{n_3=1}^{N_3} L_{sn_1} \leq \sum_{e=1}^{E} Oc_{es} \times \gamma s, \quad \forall s \in S \tag{16}$$

$$\sum_{m=1}^{M} D_{qm} \leq \left(\sum_{j=1}^{J} Gw_{jq} + \sum_{e=1}^{E} Ow_{eq}\right) \times \gamma q, \quad \forall q \in Q \tag{17}$$

$$\sum_{k=1}^{K} P_{kn_1} \geq Dp_{n_1}, \quad \forall n_1 \in N_1 \tag{18}$$

$$\sum_{e=1}^{E} O_{en_2} \geq Du_{n_2}, \quad \forall n_2 \in N_2 \tag{19}$$

$$\sum_{s=1}^{S} L_{sn_3} \geq Ds_{n_3}, \quad \forall n_3 \in N_3 \tag{20}$$

$$\sum_{q=1}^{Q} D_{qm} \leq Dc_m, \quad \forall m \in M \tag{21}$$

- The objective function (1) minimizes the total costs of the supply chain, including opening, transportation, and production costs. The costs are divided into three components: z_1 (22), z_2 (23), and z_3 (24), which represent the costs of opening facilities, transportation, and production, respectively as shown in the following

equations:

$$z_1 = \sum_{j=1}^{J} Fu_j \times U_j + \sum_{q=1}^{Q} Fy_q \times Y_q + \sum_{k=1}^{K} Fw_k \times W_k + \sum_{e=1}^{E} Fr_e \times R_e + \sum_{s=1}^{S} Fv_s \times V_s \tag{22}$$

$$\begin{aligned} z_2 = & \sum_{i=1}^{I}\sum_{j=1}^{J} CI_i \times X_{ij} + \sum_{j=1}^{J}\sum_{k=1}^{K} Cu_{1j} \times Go_{jk} + \sum_{j=1}^{J}\sum_{e=1}^{E} Cu_{2j} \times Gr_{je} \\ & + \sum_{q=1}^{Q}\sum_{m=1}^{M} Cy_q \times D_{qm} + \sum_{k=1}^{K}\sum_{n_1=1}^{N_1} Cw_k \times P_{kn_1} \\ & + \sum_{e=1}^{E} Cr_e \left(\sum_{n_2=1}^{N_2} O_{en_2} + \sum_{s=1}^{S} Oc_{es} \right) + \sum_{s=1}^{S}\sum_{n_3=1}^{N_3} Cv_s \times L_{sn_3} \end{aligned} \tag{23}$$

$$\begin{aligned} z_3 = & \sum_{i=1}^{I}\sum_{j=1}^{J} CX_{ij} \times X_{ij} + \sum_{j=1}^{J}\sum_{k=1}^{K} CK_{jk} \times Go_{jk} + \sum_{j=1}^{J}\sum_{q=1}^{Q} CJ_{jq} \times Gw_{jq} \\ & + \sum_{e=1}^{E}\sum_{s=1}^{S} CS_{es} \times Oc_{es} + \sum_{e=1}^{E}\sum_{n_2=1}^{N_2} CN_{en_2} \times Oc_{en_2} + \sum_{e=1}^{E}\sum_{q=1}^{Q} CQ_{eq} \times Ow_{eq} \\ & + \sum_{s=1}^{S}\sum_{n_3=1}^{N_3} Cl_{sn_3} \times L_{sn_3} + \sum_{k=1}^{K}\sum_{n_1=1}^{N_1} Cp_{kn_1} \times P_{kn_1} + \sum_{q=1}^{Q}\sum_{m=1}^{M} Cd_{qm} \times D_{qm} \end{aligned} \tag{24}$$

- The variables R_j, Y_q, W_k, Z_e, and V_s are binary decision variables that indicate whether certain elements in the model are active. For example, $R_j = 1$ if facility j is open, and 0 otherwise. The other variables in the model are continuous and positive, representing real numbers greater than or equal to zero, used for modeling flows, quantities, and other measurable factors.
- The constraints (2)-(7) address the finite capacities and demands of each center within the supply chain. Specifically, constraint (2) limits the quantity of products shipped from producers to distribution centers. Constraints (3)-(7) then ensure that the quantities of products delivered to processing centers, compost centers, pistachio factories, oil extraction centers, and cosmetic factories do not exceed the capacities of these centers.
- Products sent from any center must not exceed the orders received by that center. Therefore, constraint (8) stipulates that the quantity of products shipped from a processing center must be less than or equal to the quantity delivered to other centers. Constraints (9)-(12) specify the quantities of open-mouth pistachios, raw kernels, and waste produced through the dehulling process at each processing center. Similarly, constraints (13)-(17) regulate the flows within pistachio factories, oil extraction centers, cosmetic factories, and composting centers.
- Finally, to ensure that customer demand is met, constraints (18)-(21) guarantee that the products delivered to pistachio customers, pistachio oil customers, cosmetic customers, and compost markets meet or exceed their respective demands.

3 Heuristic approach

This section outlines the methodology for solving the optimization problem, combining priority-based encoding [21] and VNS. Priority-based encoding ranks decision variables, enabling the exploration of the solution space while maintaining feasibility. The VNS algorithm systematically shifts between neighborhood structures to escape local optima and explore diverse solution landscapes. The approach includes tailored neighborhood operators and specific parameter configurations.

3.1 Solution representation

As discussed in [10], this work also employs the priority-based encoding scheme to represent the solution space. Following the approach proposed by Gen, Altiparmak, and Lin [21] in their application of Genetic Algorithms (GA) to a simpler version of the problem, each source-depot pair in the supply chain is treated as a gene within the chromosome. The priority of each source-depot pair is encoded in the chromosome, which then guides the construction of the transportation tree.

The decoding process starts by identifying the node with the highest priority in the chromosome and connecting it to the corresponding node with the lowest transportation cost. The shipment amount is determined by the minimum of the source's capacity and the depot's demand. Once the shipment is fulfilled, the depot's priority is updated (or set to zero if its demand is satisfied), and the process continues with the next highest priority node. This sequential process of arc appending and priority updating repeats until all depots' demands are satisfied, resulting in a complete transportation tree.

It is important to note that, for the problem described by equations (1)-(21), each source-depot pair in the chain has its own priority sequence, which must be respected during the construction of the transportation tree. Therefore, the solution as a whole is represented by a set of chromosomes, or more generally, vectors, where each one contains the priority sequence of a source-depot pair. In [10], each element in the vector is a number between 0 and 1, and the priority among these elements is defined by their ascending order. In our work, we consider integer numbers from 1 to Z, where Z is the number of source-depot pairs in each segment of the chain. The priority among these elements is defined by their ascending numerical order.

The representation using real numbers between 0 and 1 is more suitable for handling traditional crossover and mutation operators, as is the case in [10]. However, the representation using integers from 1 to Z is more intuitive and facilitates the implementation of specific neighborhood operators, as we will discuss later. It is also worth noting that, in the case of values between 0 and 1, small perturbations in these values may not change the relative order of the elements, resulting in the same solution. This does not occur when working with integer numbers.

The effectiveness of using chromosome decoding in supply chain optimization has been well-documented in recent literature. For instance, this approach has been applied successfully to the design of integrated forward/reverse logistics networks [17], multi-objective supply chain network designs [22], and dynamic scheduling and routing

problems in flexible manufacturing systems [23]. These studies highlight the versatility and effectiveness of this decoding method in handling complex supply chain problems, including optimizing costs, responsiveness, and resource allocation across various supply chain configurations.

The decoding process is explained in [21] for a basic modeling of the supply chain problem. However, the authors of [10], who employ this methodology, did not clarify how they adapted the decoding process from [21] to the problem described by (1)-(21), which involves significantly more constraints. This description is crucial because it is directly related to ensuring solution feasibility. Without it, a fair comparison of algorithms cannot be guaranteed. Therefore, we present as follows the methodology for adapting the decoding process to the problem at hand.

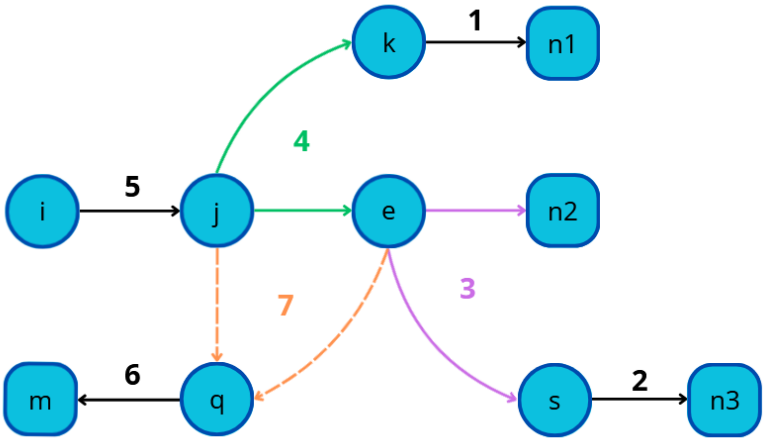


Fig. 2: Decoding steps of the transportation tree

As shown in Figure 2, the decoding process begins with the flow of pistachio products from factories to consumers (18), ensuring that all constraints are met: consumer demand is satisfied, the total amount of product leaving the factory does not exceed the pistachio received from processing centers multiplied by the factory's production rate, and the pistachio sent to any active factory is within its capacity. Likewise, the distribution of cosmetic products from factories to consumers (20) and the transfer of oil from extraction centers to both oil consumers and cosmetic factories (19) adhere to the same principles, ensuring all constraints are satisfied.

The flow from processing centers to pistachio factories and oil extraction centers is regulated by constraints (8)-(10) ensuring that the total pistachios leaving a processing center do not exceed the harvested amount sent by producers. Harvested pistachios are divided into three parts, each assigned to a product type, with a $(1 - \beta)$ loss factor applied to pistachios with shells and pits. Factories can receive more pistachios than they process, but must meet a minimum intake, while oil extraction centers send no more than $(1 - \lambda)$ of the kernels to consumers and cosmetics factories (14), with

the remaining λ going to composting (15). All processing centers must respect their capacity limits to ensure efficient distribution.

The decoding process for this specific flow is more complex and requires a targeted approach. First, we must determine the flow of in-shell pistachios from processing centers to pistachio factories, and the flow of kernels from processing centers to oil extraction centers. Using the given constraints (13)-(14), we calculate the quantity of harvested pistachios each processing center must receive to meet the demands of both pistachio factories and oil extraction centers.

If the factory demand exceeds that of the oil extraction centers, the processing center must prioritize meeting the exact demand of the factories. Since harvested pistachios are divided into three proportional parts (12), basing the allocation solely on the oil extraction center's demand would leave the factories undersupplied. The priority should be given to the entity with the greatest need, adjusting the remaining flow accordingly, as constraints allow for over-supply but not shortages.

Thus, we need to adjust the quantity of kernels shipped from the processing centers to the oil extraction centers, ensuring enough is sent to satisfy the restrictions. This involves calculating the total amount of kernels required, identifying the oil extraction center with the lowest transportation cost, and directing the additional flow there to minimize costs. If, instead, the oil extraction centers have higher demand, the process is reversed: the pistachio factories' flow is adjusted accordingly to balance the supply while minimizing transportation costs.

Next, the flow of harvested pistachios from producers to processing centers is also governed by constraints (2)-(3) ensuring that no producer exceeds its production capacity and no processing center receives more than it can process. While the capacity of each producer limits the supply, the demand for each processing center has already been determined in a previous step. If producers' capacities are insufficient to meet the total demand, the solution becomes infeasible. Therefore, it depends on the feasibility of the problem, i.e., if the parameters Cpa_i and Cpu_j are such that the total demand can be met.

Similarly, the flow of fertilizer from composting centers to consumers is governed by constraints that ensure composting centers do not receive more waste than they can handle. The amount of fertilizer sent must be less than or equal to the waste received from processing and oil extraction centers, adjusted by the composting center's production rate (17). Finally, for the waste flow from processing and oil extraction centers to composting centers, constraint (4) also ensures that no composting center receives more waste than its capacity. Waste from each processing center is assigned to the composting center with the lowest associated cost, ensuring that no composting center receives more waste than required (21). If the waste sent to a composting center falls short of its needs, the demand is adjusted accordingly. Conversely, if excess waste is sent, the demand is set to zero. Once all processing centers have allocated their waste, any remaining composting needs are fulfilled by oil extraction centers, considering their available capacity. If no additional waste is needed, no waste is sent from the oil extraction centers. Finally, the total cost of transporting waste is updated based on the assignments, ensuring efficient waste management across the supply chain.

3.2 Variable Neighborhood Search

VNS is a well-known metaheuristic that combines local search with systematic changes in neighborhood structures to explore the solution space [15]. It investigates remote regions of the solution space surrounding the current best solution, transitioning to a new solution only when an improvement is identified. The local search procedure is iteratively applied to refine solutions within these neighborhoods, driving them toward local optima.

The algorithm begins by initializing a solution and setting parameters such as the maximum neighborhood size and time limit. It then follows a cycle of shaking and local search procedures to generate new candidate solutions. If an improvement is found, the current solution is updated, and the neighborhood size is reset. Otherwise, the neighborhood size is increased, allowing the search to explore broader areas of the solution space. This process continues until the time limit is reached, at which point the best solution found is returned. Algorithm 1 provides a detailed outline of VNS.

This structure is well-suited for handling problems with multiple constraints and complex solution spaces, as shown by its application in various fields. For example, VNS has been applied to vehicle routing problems with heterogeneous fleets, successfully balancing routing costs and fleet diversity while setting new benchmarks [24]. It has also been used to solve the capacitated p-median problem by minimizing dissimilarity within clusters while adhering to capacity constraints [25]. These cases underscore VNS's flexibility in addressing constrained optimization challenges, similar to those faced in supply chain management. Hybrid approaches combining VNS with other methods, like Integer Programming, have further demonstrated its ability to refine solutions for highly constrained problems such as nurse rostering [26].

Algorithm 1 Variable Neighborhood Search (VNS)

Require: Initial solution x, maximum neighborhood size $k_{\max}$, time limit $t_{\max}$
Ensure: Best found solution x^*

$x^* \Leftarrow x$
repeat
  $k \Leftarrow 1$
  repeat
    $x' \Leftarrow \text{Shake}(x, k)$ ▷ Shaking
    $x'' \Leftarrow \text{LocalSearch}(x')$ ▷ Local search
    if $f(x'') < f(x^*)$ **then**
      $x^* \Leftarrow x''$
      $k \Leftarrow 1$ ▷ Reset neighborhood size
    else
      $k \Leftarrow k + 1$
    end if
  until $k > k_{\max}$
  $t \Leftarrow \text{CpuTime}()$
until $t > t_{\max}$

3.3 Neighborhood structures

In the context of VNS, neighborhood operators play a crucial role in exploring the solution space by systematically altering the current solution to generate new candidate solutions. The effectiveness of VNS relies heavily on the design and application of these operators, as they determine the diversity and direction of the search process. In this study, we utilize four classic neighborhood operators: Swap, Reversion, Insertion, and Slide [8]. Each operator modifies the solution in a distinct manner, allowing the algorithm to escape local optima and explore different regions of the search space. By iteratively applying these operators, VNS enhances its capability to find high-quality solutions through continuous diversification and intensification phases. The neighborhood structures used are detailed below, and illustraed in Figure 3.:

- **Swap:** The swap operator exchanges the positions of two selected elements within the chromosome. In the example, the selected elements are 6 and 3. This operator is useful for making quick adjustments to the order of elements without altering the overall structure significantly.
- **Reversion:** The reversion operator reverses the order of all elements between the two selected indices. In this case, with elements 6 and 3 chosen, the operator reverses the sequence between these two points. This operation can effectively rearrange subsequences, potentially moving the solution closer to a local optimum by drastically altering a segment of the sequence. [27]
- **Insertion:** In this operation, the unit in the second selected position is relocated immediately after the unit in the first position. Subsequently, all units that were originally between these positions are shifted accordingly to the right. This allows the algorithm to make focused adjustments by repositioning a specific element within the sequence while maintaining the overall order of the other elements. [28]
- **Slide:** This operator begins by selecting two units of a solution randomly, like the other three. The first unit is removed, and the units that were initially located between the two selected units are shifted toward the start position of the first unit. Finally, the initially removed unit is placed at the position of the second unit. This sliding mechanism creates a subtle yet impactful reordering of the solution elements, which can enhance the search process by generating new configurations with minimal disruption.

It is important to emphasize that these operations are performed considering a priority vector within the set that characterizes a solution. Therefore, a priority vector, which represents a source-depot pair in the chain, is randomly selected, and one of these operations is applied to it.

4 Computational Experiments

The proposed method is tested on a set of instances with varying sizes. The size of each instance is defined by the number of elements in each entity of the problem, namely I, J, K, E, S, Q, N_1, N_2, N_3, and M. To construct the instances, values of 100, 200, 400, 800, and 1600 are considered for each of these entities. This means that the size of each entity is uniform across all of them, resulting in 100,500; 401,000; 1,602,000;

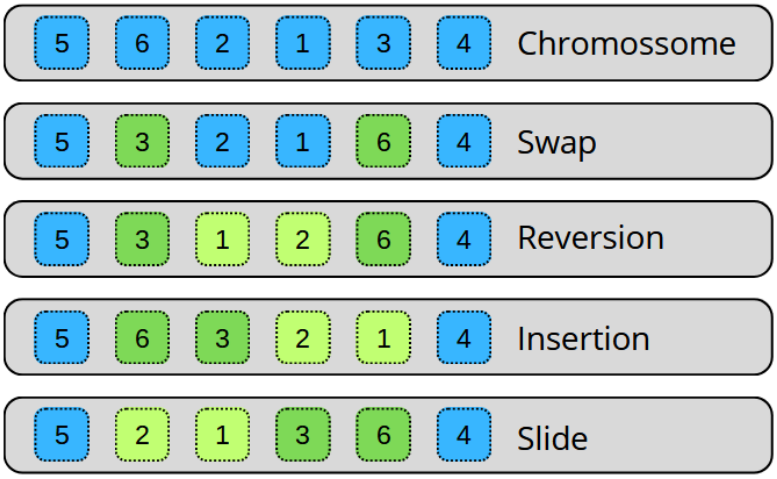


Fig. 3: Neighborhood structures used in the VNS Algorithm

6,404,000; and 25,608,000 variables for each instance, respectively. Each instance is referenced by its base number of entities, i.e., 100, 200, 400, 800, and 1600.

It is important to note that algorithms using the priority-based representation have a significantly smaller number of variables. This is because the total number of priorities is the sum of the priorities for each source-depot pair. In contrast, the mathematical model (1)-(21) has a number of variables equal to the sum of all possible transportation options for each source-depot pair, plus the binary variables indicating whether each entity is active or not. For the priority-based algorithms, the number of variables is 1,700; 3,400; 6,800; 13,600; and 27,200, respectively.

The other parameters of the model, such as capacities and demands, were randomly defined for each instance based on the information provided in Table 7 of [10]. The proposed method is compared with two other approaches: a Genetic Algorithm (GA) and an exact method for Mixed-Integer Linear Programming (MILP) implemented by Gurobi 11.0.3 [29]. The GA was implemented with mutation operators based on swapping elements within a priority vector and crossover operators based on exchanging subsequences of priorities between two vectors. In both cases, the vectors are selected randomly. The population size was set 100 and the Binary Tournament was applied as selection strategy.

The stopping criterion for the proposed method and the GA was set to 100,000 evaluations. For each instance, the proposed method and the GA were executed 30 times. The experiments were conducted on a machine with the following configuration: Ubuntu 24.04.1 LTS, Intel Xeon E3-1240 V2 with 8 cores, and 32 GB of RAM.

4.1 Comparison between VNS and GA

The objective of this experiment was to compare the proposed methodology with a method similar to those used in the primary reference [10]. However, a fair comparison

with the algorithms from [10] was not feasible, as the decoding methodology was not explicitly presented in their work. Consequently, the results of this experiment serve to establish a relationship between the proposed method in this study and the reference, providing insights into their relative performance and applicability.

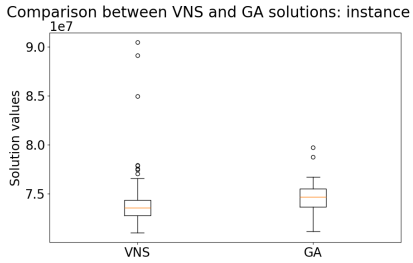
The average performance of the proposed method compared to the Genetic Algorithm (GA) is presented in Table 1, which shows the average of the 30 best solutions found for each algorithm and the average execution time. For small instances, such as the 100-one, the difference between VNS and GA is negligible, with only a 0.71% improvement. However, for larger instances, VNS demonstrates a significantly better performance, outperforming GA by 3.4 to 5.3% on average. The boxplots in Fig 4 illustrate the distribution of objective function values for each algorithm across different instance sizes. Notably, significant differences are observed in the 800 and 1600 instances. The p-values from the Independent Samples T-test (Welch Test) for these instances are less than 0.05, confirming that the differences are statistically significant under the assumptions of normality and unequal variances. These results indicate that VNS is statistically superior to GA for large-scale instances.

It was observed that the execution time of the GA was significantly longer than that of VNS. However, it was not possible to ensure that the implementations of both GA and VNS were fully optimized, making a fair comparison challenging. One possible explanation is that the evolutionary operations of the GA, such as crossover and mutation, may have become more computationally expensive as the number of variables increased substantially. This could have contributed to the higher runtime of the GA compared to VNS, particularly in larger instances.

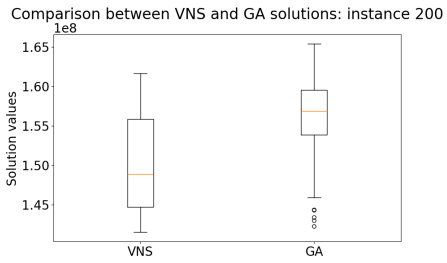
In summary, while GA might be equivalent with VNS in smaller problem instances, it lags behind as the problem increases. VNS consistently provides not only better solutions, as supported by statistical evidence, but also handles larger instances more efficiently, suggesting that it is more robust and scalable for large-scale optimization problems.

Table 1: Comparison of the VNS algorithm and a population-based metaheuristic (Genetic Algorithm) in terms of the average objective function value of the best solutions and the average execution time.

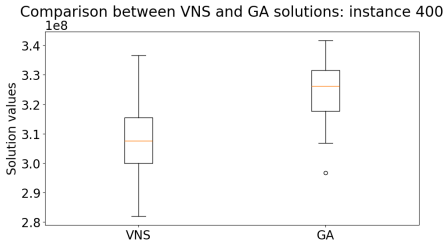
	Objective function evaluation (average)			Execution time (average)	
Instance	VNS	GA	Difference	VNS	GA
100	7.36×10^6	7.46×10^7	0.71%	241s	6,595s
200	1.49×10^8	1.55×10^8	3.37%	714s	14,349s
400	3.08×10^8	3.24×10^8	5.31%	2,016s	33,941s
800	6.57×10^8	6.92×10^8	5.26%	5,538s	82,687s
1600	1.36×10^9	1.43×10^9	4.64%	17,767s	217,958s



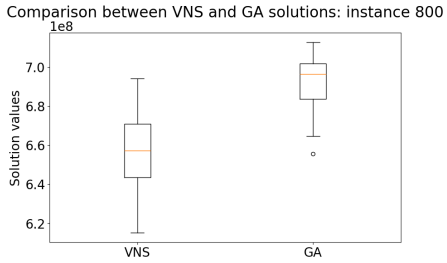
(a) I, J, K, E, S, Q, N1, N2, N3, M = 100



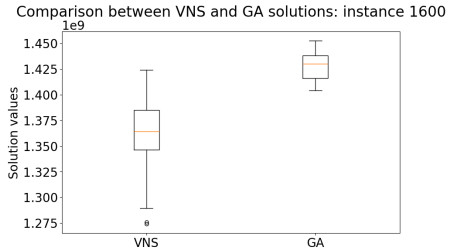
(b) I, J, K, E, S, Q, N1, N2, N3, M = 200



(c) I, J, K, E, S, Q, N1, N2, N3, M = 400



(d) I, J, K, E, S, Q, N1, N2, N3, M = 800



(e) I, J, K, E, S, Q, N1, N2, N3, M = 1600

Fig. 4: Boxplot visualizations comparing the performance of the VNS algorithm and a population-based metaheuristic (genetic algorithm) across different problem sizes, based on objective function values.

4.2 Comparison between VNS and an Exact Algorithm

This experiment focuses on a comparison between the proposed VNS and an exact MILP algorithm. The goal is to evaluate the effectiveness of VNS in finding high-quality solutions compared to an exact algorithm, which guarantees optimal solutions. The comparison is conducted in terms of the average objective function values of the best solutions found and the average execution time. By analyzing these metrics, we aim to assess the trade-off between solution quality and computational efficiency,

particularly in scenarios where the MILP approach may become computationally prohibitive for larger instances. In fact, the MILP approach was unable to solve the 1600 instance due to memory constraints and it terminated the 800 instance prematurely due to the same memory constraints. This implies that the solution found for the 800 instance might not be the global optimum.

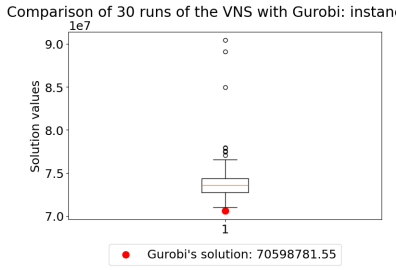
As shown in Table 2, the exact method efficiently finds optimal solutions for smaller instances, such as the 100 and 200 ones. In these cases, the difference between the average objective function values of VNS and exact method is about 4.19% for the 100 instance set and 8.22% for the 200 instance set. However, as the problem size increases, the gap in solution quality widens significantly, with VNS being 20.12% from the optimal solution for the 800 instance set.

The boxplots in Figure 5 further illustrate the variability of VNS across 30 runs compared to exact method's single optimal solution. For 100-instance, VNS results range from 7.15×10^6 to 8.75×10^6, closely matching exact method's solution of 7.06×10^6 with only a 4.19% difference in the best case. In instance 200, VNS shows slightly more variability, ranging from 1.40×10^8 to 1.75×10^8, maintaining an 8.22% gap from Gurobi's 1.37×10^8, in average. As problem size increases, the VNS range widens – for 400-instance, from 2.70×10^8 to 3.30×10^8, with a 13.62% difference in the best case, and for 800-instance, from 5.40×10^8 to 6.85×10^8, showing a 20.12% gap from Gurobi's 5.47×10^8, in the best case.

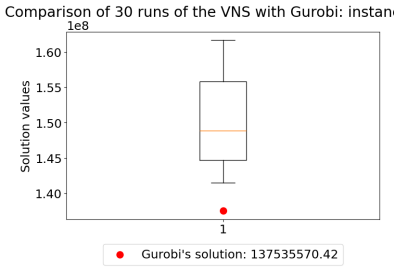
When examining execution time, the computational cost of the exact method grows significantly. Since the exact method was unable to address the 1600 instance and could not complete the execution for the 800 instance, this represents a significant drawback for large-scale problems. In these situations, VNS might be the preferred option to address such problems, as it can find reasonable solutions with a much lower computational cost. It is important to note that the quality of the final solution and the execution time are strongly related to the stop criterion, which was the same for all instances. Therefore, better solutions could potentially be found by increasing the number of evaluations, but at the cost of longer execution times. It is also important to note that many instances in the computational experiments in [10] are significantly smaller than the ones used in this study. In such cases, the use of a heuristic is not justifiable, as the exact method can solve the problem in a reasonable time.

Table 2: Comparison of the VNS algorithm and the Exact Algorithm (Gurobi) in terms of the average objective function value of the best solutions and the average execution time.

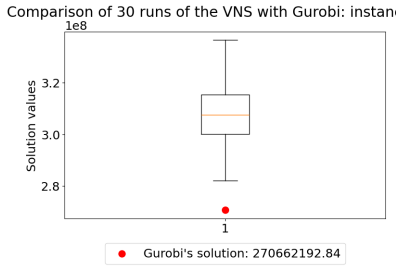
	Objective function evaluation (average)			Execution time (average)		
Instance	VNS	Exact Algorithm	Difference	VNS	Exact Algorithm	Difference
100	7.36×10^6	7.06×10^6	4.19%	241s	261s	-7.83%
200	1.49×10^8	1.38×10^8	8.22%	714s	3834s	-81.36%
400	3.08×10^8	2.71×10^8	13.62%	2016s	70339s	-97.13%
800	6.57×10^8	5.47×10^8	20.12%	5538s	11836s	-53.22%
1600	1.36×10^9	-	-	17767s	-	-



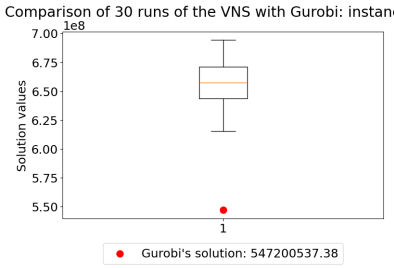
(a) I, J, K, E, S, Q, N1, N2, N3, M = 100



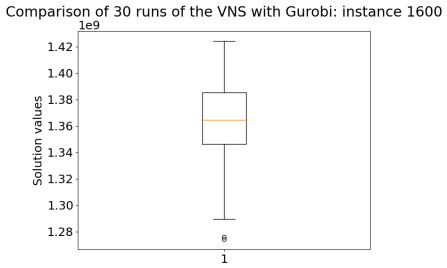
(b) I, J, K, E, S, Q, N1, N2, N3, M = 200



(c) I, J, K, E, S, Q, N1, N2, N3, M = 400



(d) I, J, K, E, S, Q, N1, N2, N3, M = 800



(e) I, J, K, E, S, Q, N1, N2, N3, M = 1600. The exact algorithm was not able to return a solution due to excessive resource demands.

Fig. 5: Boxplot visualizations comparing the performance of the VNS algorithm and an exact algorithm (Gurobi) across different problem sizes, based on objective function values.

5 Conclusion

This study addresses a comprehensive optimization model for the pistachio supply chain network, addressing the complexities of managing pistachio products and by-products across multiple stages. By integrating production, transportation, and facility

operations, the model offers a holistic approach to supply chain management, ensuring that all capacity and demand constraints are met. The model is formulated as a MILP problem, which is then solved by the proposed formulation of VNS algorithm. The proposed VNS is formulated with four neighborhood structures, namely Swap, Reversion, Insertion, and Slide, which are applied iteratively to explore the solution space and refine solutions.

Other works in the literature have already addressed this problem, particularly using population-based metaheuristics such as GA. When compared to GA, VNS demonstrated superior performance in terms of both solution quality and computational efficiency. The proposed method is especially advantageous for large-scale optimization problems, where the exact method becomes impractical due to memory constraints and prohibitively long execution times. However, for small or medium-sized instances, the exact method remains the best option, as it guarantees optimal solutions.

Moving forward, research could benefit from the development of more specialized and problem-specific neighborhood operators for VNS, which may lead to even better performance by enabling more targeted exploration of the solution space. Additionally, combining VNS with exact algorithms for smaller problem instances presents an intriguing opportunity to balance accuracy and efficiency, creating a math-heuristics that could be more broadly applicable across industries. The repository with the code and instances used in this study is available at https://github.com/raissagd/Pistachio_CLSC.

References

[1] Statista. Leading producers of pistachios worldwide in 2022 and 2023 (2023). URL https://www.statista.com/statistics/933042/global-pistachio-production-by-country/. Accessed: 2023-10-05.

[2] Doula, M., Kouloumbis, P. & Elaiopoulos, K. Turning wastes into valuable materials: Valorization of pistachio wastes in agricultural sector 19–21 (2014).

[3] Bartzas, G. & Komnitsas, K. Life cycle analysis of pistachio production in greece. *Science of the Total Environment* **595**, 13–24 (2017).

[4] Hokmabadi, H. Pistachio wastes in iran and the potential to recapture them in value chain. *Pistachio and Health Journal* **1**, 1–12 (2018).

[5] Dronavalli, S. & Kang, M. S. Microgametophytic selection for identifying maize with resistance to aspergillus spp. and aflatoxin. *Journal of Crop Improvement* **33**, 363–371 (2019).

[6] Casper, R. & Sundin, E. Reverse logistic transportation and packaging concepts in automotive remanufacturing. *Procedia Manufacturing* **25**, 154–160 (2018).

[7] Cheraghalipour, A. & Roghanian, E. A bi-level model for a closed-loop agricultural supply chain considering biogas and compost. *Environment, Development*

and Sustainability 1–47 (2022).

[8] Devika, K., Jafarian, A. & Nourbakhsh, V. Designing a sustainable closed-loop supply chain network based on triple bottom line approach: A comparison of metaheuristics hybridization techniques. *European journal of operational research* **235**, 594–615 (2014).

[9] Zohal, M. & Soleimani, H. Developing an ant colony approach for green closed-loop supply chain network design: a case study in gold industry. *Journal of Cleaner Production* **133**, 314–337 (2016).

[10] Rajabi-Kafshgar, A., Gholian-Jouybari, F., Seyedi, I. & Hajiaghaei-Keshteli, M. Utilizing hybrid metaheuristic approach to design an agricultural closed-loop supply chain network. *Expert Systems with Applications* **217**, 119504 (2023).

[11] Fathollahi-Fard, A. M. & Hajiaghaei-Keshteli, M. A stochastic multi-objective model for a closed-loop supply chain with environmental considerations. *Applied Soft Computing* **69**, 232–249 (2018).

[12] Xie, Y., Sheng, Y., Qiu, M. & Gui, F. An adaptive decoding biased random key genetic algorithm for cloud workflow scheduling. *Engineering applications of artificial intelligence* **112**, 104879 (2022).

[13] Campelo, F. & Aranha, C. Lessons from the evolutionary computation bestiary. *Artificial Life* **29**, 421–432 (2023). URL https://doi.org/10.1162/artl_a_00402.

[14] Aranha, C. *et al.* Metaphor-based metaheuristics, a call for action: the elephant in the room. *Swarm Intelligence* **16**, 1–6 (2022).

[15] Gendreau, M., Potvin, J.-Y. *et al. Handbook of metaheuristics* Vol. 2 (Springer, 2010).

[16] Hansen, P., Mladenović, N. & Moreno Perez, J. A. Variable neighbourhood search: methods and applications. *Annals of Operations Research* **175**, 367–407 (2010).

[17] Pishvaee, M. S., Farahani, R. Z. & Dullaert, W. A memetic algorithm for bi-objective integrated forward/reverse logistics network design. *Computers & operations research* **37**, 1100–1112 (2010).

[18] Ahmadi, S. & Amin, S. H. An integrated chance-constrained stochastic model for a mobile phone closed-loop supply chain network with supplier selection. *Journal of cleaner production* **226**, 988–1003 (2019).

[19] Banasik, M., Stanisławska-Sachadyn, A. & Sachadyn, P. A simple modification of pcr thermal profile applied to evade persisting contamination. *Journal of Applied Genetics* **57**, 409–415 (2016).

[20] Gholian-Jouybari, F., Hashemi-Amiri, O., Mosallanezhad, B. & Hajiaghaei-Keshteli, M. Metaheuristic algorithms for a sustainable agri-food supply chain considering marketing practices under uncertainty. *Expert Systems with Applications* **213**, 118880 (2023).

[21] Gen, M., Altiparmak, F. & Lin, L. A genetic algorithm for two-stage transportation problem using priority-based encoding. *OR spectrum* **28**, 337–354 (2006).

[22] Altiparmak, F., Gen, M., Lin, L. & Paksoy, T. A genetic algorithm approach for multi-objective optimization of supply chain networks. *Computers & industrial engineering* **51**, 196–215 (2006).

[23] Liu, J., Sun, B., Li, G. & Chen, Y. An integrated scheduling approach considering dispatching strategy and conflict-free route of amrs in flexible job shop. *The International Journal of Advanced Manufacturing Technology* **127**, 1979–2002 (2023).

[24] Imran, A., Salhi, S. & Wassan, N. A. A variable neighborhood-based heuristic for the heterogeneous fleet vehicle routing problem. *European Journal of Operational Research* **197**, 509–518 (2009).

[25] Fleszar, K. & Hindi, K. S. An effective vns for the capacitated p-median problem. *European Journal of Operational Research* **191**, 612–622 (2008).

[26] Burke, E. K., Li, J. & Qu, R. A hybrid model of integer programming and variable neighbourhood search for highly-constrained nurse rostering problems. *European Journal of Operational Research* **203**, 484–493 (2010).

[27] Vahdani, B. & Zandieh, M. Scheduling trucks in cross-docking systems: Robust meta-heuristics. *Computers & Industrial Engineering* **58**, 12–24 (2010).

[28] Rabiee, M., Zandieh, M. & Jafarian, A. Scheduling of a no-wait two-machine flow shop with sequence-dependent setup times and probable rework using robust meta-heuristics. *International Journal of Production Research* **50**, 7428–7446 (2012).

[29] Gurobi Optimization, LLC. Gurobi Optimizer Reference Manual (2024). URL https://www.gurobi.com.